

TKV: Online KV Cache Compression via Warmup-Then-Freeze Brand Updates and a Random Sketch Bridge

Anonymous authors

Paper under double-blind review

Abstract

The key-value (KV) cache is the dominant memory bottleneck in long-context autoregressive inference. Existing online compression methods face a tradeoff: Oja’s rule (used by OjaKV) is cheap but gives no per-token optimality guarantee and drifts at long context, while Brand’s exact rank-1 SVD update is provably optimal but costs $\mathcal{O}(R^3 + TR^2)$ per token — growing with context length. We propose TKV (Tensor KV), which resolves this by running Brand’s update only through a bounded warmup window, then freezing the resulting basis and projecting subsequent tokens onto it at $\mathcal{O}(R \cdot d)$ per token indefinitely. A Φ -bridge reuses the Gaussian matrix already used for token-importance tracking as the Halko randomized range finder for cold-start initialization, and the update is implemented with XLA-static shapes for GPU/TPU compatibility. On real Mistral-7B activations, a TKV rank pyramid (avg. rank 15) reaches perplexity 20.1 versus an exact-cache baseline of 19.4 (3.6% degradation), while the same pyramid schedule applied to OjaKV degrades to $60\times$ worse than exact; at 128K-token context TKV achieves $8.5\times$ memory compression and lower reconstruction error than OjaKV at every long-context length tested (1K–32K tokens). Importance-weighted cold start further reduces important-token reconstruction error by 90–95% at rank 8–16.

1 Introduction

The key-value (KV) cache is the dominant memory bottleneck in autoregressive transformer inference: for L layers, H KV heads, head dimension d , and context length T , the dense cache requires $\mathcal{O}(2LHdT)$ floats, becoming the binding constraint on batch size, GPU utilization, and serving cost as context scales to 128K+ tokens.

The predominant approach is low-rank compression: represent the key matrix $K \in \mathbb{R}^{T \times d}$ as a rank- R factorization $U\Sigma V^\top$ with $R \ll \min(T, d)$, reducing memory from $\mathcal{O}(Td)$ to $\mathcal{O}(R(T + d))$. The key challenge is when to compute this factorization. Offline methods (SVDq (Anonymous, 2025c), xKV (Anonymous, 2025d), KVTC (Anonymous, 2026)) run a batch SVD after the full sequence is available, incompatible with streaming generation; online methods must update the factorization incrementally after each new token. OjaKV (Anonymous, 2025b), the current online state of the art, applies Oja’s rule (stochastic gradient PCA) token-by-token — asymptotically convergent but with no per-step optimality guarantee. When OjaKV’s authors considered Brand’s exact rank-1 update as an alternative, they rejected it as “too slow” without a systematic evaluation.

This work. We revisit that judgment for a bounded warmup window, then freeze the resulting basis for the remainder of the sequence rather than running Brand’s exact update ($\mathcal{O}(R^3 + TR^2)$ per token) forever, which would otherwise let its cost compound at long context (Section 4). During warmup itself, exact updates are cheap and substantially more accurate than Oja’s rule (Section 4); the freeze transition preserves this advantage while holding per-token cost at $\mathcal{O}(R \cdot d)$ indefinitely. We also identify a structural insight missed by prior work: the random Gaussian matrix used for token-importance scoring satisfies exactly the distributional requirements of Halko’s randomized SVD range finder, so reusing

it (the Φ -bridge) unifies two previously separate operations and eliminates a distinct SVD warm-up cost.

Contributions.

- The Φ -bridge: a theoretical and practical connection between random linear sketching and Halko’s randomized SVD (Section 2.2).
- To our knowledge, the first online KV compression method with a per-token EYM optimality guarantee through a warmup window, followed by a freeze transition that holds per-token cost at $\mathcal{O}(R \cdot d)$ indefinitely (Sections 2.4–2.5).
- An XLA-compilable implementation of Brand’s dynamic rank-1 update using static tensor shapes (Section 2.6).
- Importance-weighted cold start that reduces important-token reconstruction error by 90–95% at rank 8–16 (Section 2.7).
- Empirical evaluation against OjaKV under identical constraints, showing consistent quality improvement at the cost of moderate per-token overhead (Section 4).

2 Method

2.1 Preliminaries

KV cache structure and EYM. In a transformer with head dimension d , after processing t tokens the KV cache for one head is a matrix $K \in \mathbb{R}^{t \times d}$ (and similarly V); each new token appends one row, and compression approximates $K \approx \hat{K} = U_K \Sigma_K V_K^\top$ with $R \leq \min(t, d)$. By the Eckart-Young-Mirsky (EYM) theorem, the best rank- R approximation of any matrix A in Frobenius or spectral norm is its truncated SVD $\hat{A}_R = U_R \Sigma_R V_R^\top$, so any online method that maintains a valid truncated SVD after each update is EYM-optimal for that update.

Brand’s rank-1 update (Brand, 2006) and Halko randomized SVD (Halko et al., 2011). Given a factorization $A = U \Sigma V^\top$ and a new row $\mathbf{x}$, Brand’s update computes the exact SVD of $\tilde{A} = [A; \mathbf{x}]$ in closed form by factoring a small $(R+1) \times (R+1)$ augmented matrix and truncating back to rank R (an exact EYM step per token), then updating U via an $\mathcal{O}(tR^2)$ rotation (Section 2.4 gives the full algorithm). Halko’s randomized SVD instead sketches $A \in \mathbb{R}^{m \times n}$ with a random $\Omega \in \mathbb{R}^{n \times R}$: the range of $A\Omega$ approximates A ’s column space, and QR on $A\Omega$ followed by the compact SVD of $Q^\top A$ gives the randomized SVD at $\mathcal{O}(mnR)$ cost vs. $\mathcal{O}(mn^2)$ for exact SVD, with high-probability error bounds (Halko et al., 2011).

Oja’s rule (Oja, 1982). For streaming PCA, Oja’s rule updates the $R \times d$ basis V after each observation $\mathbf{x}$ as $V \leftarrow V + \eta_t (V \mathbf{x}) \mathbf{x}^\top$, $V \leftarrow \text{QR}(V)^\top$, with step size $\eta_t = 1/(t+1)$ — the update used by OjaKV. Unlike Brand’s update, this is a stochastic gradient step: each intermediate V is an approximation, not an exact truncated SVD.

2.2 The Φ -Bridge

A random Gaussian sketch tracks token importance via a random projection matrix $\Phi \in \mathbb{R}^{R \times d}$: for a token vector $\mathbf{x} \in \mathbb{R}^d$, $\Phi \mathbf{x} \in \mathbb{R}^R$ captures token statistics in compressed form. Halko’s randomized range finder requires exactly the same object — a random matrix $\Omega \in \mathbb{R}^{d \times R}$ whose product $K\Omega$ forms a randomized range basis for the row space of K .

Observation. Setting $\Omega = \Phi^\top$ reuses the same Gaussian matrix for both roles: $K\Phi^\top \in \mathbb{R}^{t \times R}$ approximates the principal right singular vectors of K while Φ simultaneously encodes token importance statistics. This unification — the Φ -bridge — works because the Gaussian distribution of Φ satisfies the requirements for both linear importance sketching (Johnson-Lindenstrauss concentration) and Halko’s range finder (sub-Gaussian rows, Theorem 1.1 of (Halko et al., 2011)); this differs from the structured hashing of the classical count-min

89 sketch (Cormode & Muthukrishnan, 2005), which shares only the general idea of random
90 linear projection for dimensionality reduction.

91 2.3 Cold Start via Halko Initialization

92 After observing a prefix of T_0 tokens, TKV initializes the SVD factorization via the random-
93 ized range finder:

Algorithm 1 Halko Cold Start with Φ -Bridge

Require: $K \in \mathbb{R}^{T_0 \times d}$, $V \in \mathbb{R}^{T_0 \times d}$, $\Phi \in \mathbb{R}^{R \times d}$

```

1:  $Y \leftarrow K\Phi^\top$                                 randomized sketch,  $\mathcal{O}(T_0 d R)$ 
2:  $Q, \_ \leftarrow \text{QR}(Y)$                             approximate range basis
3:  $B \leftarrow Q^\top K$                                 project into low-dimensional space
4:  $\hat{U}, \hat{\Sigma}, \hat{V}^\top \leftarrow \text{SVD}(B)$             small SVD,  $\mathcal{O}(R^2 d)$ 
5:  $U_K \leftarrow Q\hat{U}$ ,  $\Sigma_K \leftarrow \hat{\Sigma}$ ,  $V_K \leftarrow \hat{V}$ 
6: return  $(U_K, \Sigma_K, V_K)$ 

```

94 If importance weights $w_i \geq 0$ are available (e.g., from prior attention statistics), we apply
95 them as row weights: $Y \leftarrow \text{diag}(w)K\Phi^\top$, biasing the range finder toward the subspace of
96 high-importance tokens. This cold start opens TKV’s warmup phase: it is followed by exact
97 Brand updates (Section 2.4) for the first T_0 (“warmup_len”) tokens, after which the basis
98 is frozen (Section 2.5).

99 2.4 Streaming Brand Updates (Warmup Phase)

100 During warmup, each new token $(\mathbf{k}, \mathbf{v})$ triggers one Brand rank-1 update:

Algorithm 2 Brand Rank-1 Update (per token)

Require: (U_K, Σ_K, V_K) , new key $\mathbf{k} \in \mathbb{R}^d$, rank budget R

```

1:  $\mathbf{c} \leftarrow V_K \mathbf{k}$                                 project onto current row space
2:  $\mathbf{p} \leftarrow \mathbf{k} - V_K^\top \mathbf{c}$ ,  $\hat{\mathbf{p}} \leftarrow \mathbf{p} / \|\mathbf{p}\|$ 
3: Form  $K_{\text{aug}} \in \mathbb{R}^{(R+1) \times (R+1)}$  with diagonal  $\sigma$ , row extension  $\mathbf{c}$ , and corner  $\|\mathbf{p}\|$ 
4:  $\hat{U}, \hat{\Sigma}, \hat{W}^\top \leftarrow \text{SVD}(K_{\text{aug}})$ 
5:  $\sigma \leftarrow \hat{\Sigma}_{:,R}$                                 truncate to rank  $R$  (EYM step)
6: Extend  $V_K$  with  $\hat{\mathbf{p}}$ , rotate:  $V_K \leftarrow \hat{W}_{:,R}[V_K; \hat{\mathbf{p}}^\top]$ 
7: Append new  $U_K$  row, rotate:  $U_K \leftarrow [U_K; \mathbf{0}^\top] \hat{U}_{:,R,R}$ 
8: return  $(U_K, \sigma, V_K)$ 

```

101 The truncation at step 5 is exactly an EYM projection: the resulting $\hat{U}_{:,R} \hat{\Sigma}_{:,R} \hat{W}_{:,R}^\top$ is the best
102 rank- R approximation of $[K; \mathbf{k}^\top]$ in Frobenius norm. This guarantee holds after every token
103 during the warmup phase, i.e., for $t \leq T_0$; Section 2.5 describes the post-warmup regime.

104 2.5 Freeze Transition (Decode Phase)

105 Once $t = T_0$, TKV freezes the basis (V_K, V_V) produced by the most recent Brand update
106 and stops running Algorithm 2. Every subsequent token is instead projected onto this fixed
107 basis:

Algorithm 3 TKV Frozen-Phase Token Update (per token, $t > T_0$)

Require: Frozen basis $V_K \in \mathbb{R}^{R \times d}$, stored projections $\text{proj}_K \in \mathbb{R}^{(t-1) \times R}$, new key $\mathbf{k} \in \mathbb{R}^d$
 1: $\mathbf{c} \leftarrow V_K \mathbf{k}$ project onto the frozen basis, $\mathcal{O}(Rd)$
 2: Append $\mathbf{c}$ to proj_K no SVD, no rotation
 3: return proj_K

108 This is implemented by `tkv_add_token` in our reference implementation. The motivation is
 109 cost: Brand’s update is $\mathcal{O}(R^3 + TR^2)$ per token, so running it indefinitely makes per-token
 110 cost grow with context length. Freezing after a bounded warmup caps per-token cost at
 111 $\mathcal{O}(Rd)$ for the rest of the sequence, regardless of how long it runs — the gap between the
 112 two reaches $13.4\times$ by $T = 16,000$ (Section 4).

113 The honest tradeoff (stated precisely in Section 3) is that frozen-phase tokens are no longer
 114 individually EYM-optimal, since V_K was fit to K_{T_0} , not the full sequence. Empirically this
 115 is favorable regardless: OjaKV’s basis keeps rotating and its projection error compounds
 116 with t , while TKV’s frozen projection error does not, so TKV’s fixed-but-accurate basis
 117 still outperforms OjaKV’s continuously-drifting one at every long-context length we tested
 118 (Section 4).

119 2.6 XLA-Static Shape Implementation

120 Brand’s update is inherently dynamic (U_K grows by one row per token), but static-shape
 121 XLA compilation requires a fixed pre-allocated buffer $\hat{U} \in \mathbb{R}^{T_{\max} \times R}$ with the logical se-
 122 quence length tracked separately; new rows are written via `jax.lax.dynamic_update_slice`,
 123 which supports dynamic-index writes while keeping a static shape graph, and the $\mathcal{O}(TR^2)$
 124 U-rotation operates on this buffer bounded by $T_{\max}$. To our knowledge, no prior work has
 125 implemented Brand updates in XLA-compilable form, a prerequisite for GPU/TPU deploy-
 126 ment.

127 2.7 Importance-Weighted Cold Start

128 During autoregressive generation, attention weight statistics accumulate per token position.
 129 Following Moment-KV (Anonymous, 2024b), we maintain an EMA importance score $I_i \leftarrow$
 130 $\alpha I_i + (1 - \alpha) \cdot \bar{w}_i$, where $\bar{w}_i = \frac{1}{n_q} \sum_q \text{softmax}(\mathbf{q}K^\top / \sqrt{d})_{q,i}$ is the mean attention weight token
 131 i receives across queries. When the context must be re-compressed (e.g., at context-length
 132 boundaries), these scores serve as row weights in the Halko cold start, biasing the initial basis
 133 toward tokens that are actually attended to. Section 4 shows this reduces important-token
 134 reconstruction error by 90–95% at rank 8–16.

135 2.8 Compressed Attention

Attention can be computed directly from the SVD factors without explicit reconstruction
 of K or V :

$$\mathbf{s} = \mathbf{q}K^\top = \mathbf{q}V_K^\top \Sigma_K U_K^\top \in \mathbb{R}^T, \quad \mathbf{out} = \text{softmax}(\mathbf{s})U_V \Sigma_V V_V \in \mathbb{R}^d, \quad (1)$$

136 requiring only matrix-vector products with the rank- R factors ($\mathcal{O}(R(T+d))$ vs. $\mathcal{O}(Td)$ for
 137 dense attention). In the frozen phase, the same formula applies with $U_K \Sigma_K$ replaced by the
 138 stored projections proj_K , so compressed attention is computed identically before and after
 139 the freeze.

140 3 Theoretical Analysis

141 The following results follow directly from applying Brand’s exact rank-1 update iteratively
 142 under the Eckart-Young-Mirsky theorem; the contribution is identifying that this combi-
 143 nation maintains EYM-optimality token-by-token in the KV cache setting, a property that
 144 does not hold for Oja’s rule.

Corollary 1 (Per-Token EYM Optimality, Warmup Phase). Let $K_t \in \mathbb{R}^{t \times d}$ be the key matrix after $t \leq T_0$ tokens (i.e., during the warmup window), and let (U_t, σ_t, V_t) be the TKV state after processing the t -th token via Algorithm 2. Then $\|K_t - U_t \text{diag}(\sigma_t) V_t^\top\|_F = \min_{\text{rank}(B) \leq R} \|K_t - B\|_F$ (the Brand update at step t truncates the exact SVD of $K_{\text{aug}} = [K_{t-1}; \mathbf{k}_t^\top]$ to rank R , which by EYM is the best such approximation).

Proposition 1 (Post-Freeze Cost and Quality). For $t > T_0$, TKV’s basis $(V_{T_0, K}, V_{T_0, V})$ is held fixed and each new token is processed in $\mathcal{O}(Rd)$ time via projection onto it (Algorithm 3), with no further EYM updates: the projection is exact within the rowspace of $V_{T_0, K}$ but not in general the EYM-optimal rank- R approximation of K_t , since $V_{T_0, K}$ was optimized only for K_{T_0} . Quality after freezing is therefore bounded by how well the warmup-window subspace continues to span later tokens’ directions, not by a fresh per-token optimality guarantee.

Corollary 2 (Comparison to OjaKV). For $t \leq T_0$, TKV achieves the minimum Frobenius reconstruction error among all rank- R factorizations of K_t , while OjaKV’s stochastic-gradient state has no analogous guarantee and can exceed the EYM optimum at any finite t . For $t > T_0$ neither method retains a strict guarantee (Proposition 1), but empirically TKV’s fixed-but-accurate basis still outperforms OjaKV’s continuously-drifting one at every long-context length tested, since OjaKV’s staleness error compounds with t while TKV’s frozen-basis error does not (Section 4).

Proposition 2 (Cold Start Error Bound). After the Halko cold start with oversampling parameter p , the TKV approximation satisfies, with probability at least $1 - \delta$, $\|K - \hat{K}\|_F \leq \left(1 + \sqrt{R/(p+1)} \cdot t_\delta\right) \sigma_{R+1}(K)$, where σ_{R+1} is the $(R+1)$ -th singular value of K and t_δ is a tail factor from the Halko error bound (Theorem 1.1 in (Halko et al., 2011)). Brand updates maintain this quality exactly through $t = T_0$ (Proposition 1 covers the regime beyond).

4 Experiments

Real-model experiments (reconstruction, BrandOnly-vs-OjaKV head-to-head, perplexity, decode latency) use real K/V activations captured from Mistral-7B-v0.1 on WikiText-2 via a lightweight passthrough hook, with no compression applied during extraction; synthetic-data experiments (memory formula, long-context reconstruction, importance weighting) use $d = 64$ throughout, independent of the deployment model. All experiments use JAX 0.4 and $T_{\text{max}} = 1024$ where applicable; timings are Python/eager mode on CPU (GPU numbers would differ due to XLA compilation benefits).

4.1 Reconstruction Error vs OjaKV: The BrandOnly Ablation

We first establish the quality ceiling achievable by exact Brand updates with no freeze (BrandOnly) against OjaKV (Oja’s rule with $\eta_t = 1/(t+1)$) at matched sequence lengths and rank budgets — this is the warmup-phase mechanism TKV inherits, run here to its full extent (never frozen) as an ablation.

BrandOnly is strictly better in quality than OjaKV at every measured setting, largest at short contexts (up to $4.69\times$ at $T = 64$, $R = 32$) and increasing with rank, consistent with the EYM guarantee. A 12-configuration sweep (layers 0, 8, 16, $24 \times$ KV heads 0, 3, 6 of Mistral-7B; full numbers in Appendix A) confirms this is not an artifact of one head: BrandOnly dominates at every configuration, with wide advantage variance at $R = 32$ reflecting per-head singular-value-decay heterogeneity.

Speed. BrandOnly is $3.7\text{--}4.1\times$ slower per token than OjaKV in Python/eager mode (dominated by call overhead, not compute) due to the $\mathcal{O}(T_{\text{max}}R^2)$ U-rotation; the FLOPs ratio is more representative of compiled performance, at only $\sim 3.9\times$ raw overhead, falling to $\sim 0.6\times$ (cheaper than OjaKV) once the U-rotation is lazily flushed every $N=8$ tokens (Section 2.6), at the cost of bounded inter-flush error in U .

Streaming error accumulation. Figure 1 (center panel) shows how reconstruction error grows as tokens accumulate at rank 16. OjaKV’s error rises faster because Oja’s stale

Table 1: Relative Frobenius reconstruction error on Mistral-7B (layer 16, KV head 0). BrandOnly consistently achieves lower error than OjaKV; advantage grows at shorter contexts and higher ranks (per-token update time discussed in the Speed paragraph below).

T	R	BrandOnly error	OjaKV error	BrandOnly advantage
64	4	0.4823	0.7851	$1.63\times$
64	8	0.4067	0.7841	$1.93\times$
64	16	0.3012	0.7831	$2.60\times$
64	32	0.1670	0.7830	$4.69\times$
128	8	0.4499	0.7877	$1.75\times$
128	16	0.3600	0.7850	$2.18\times$
128	32	0.2427	0.7866	$3.24\times$
256	16	0.4121	0.8267	$2.01\times$
256	32	0.3010	0.8263	$2.75\times$
512	8	0.5212	0.8720	$1.67\times$
512	16	0.4466	0.8688	$1.95\times$
512	32	0.3442	0.8673	$2.52\times$

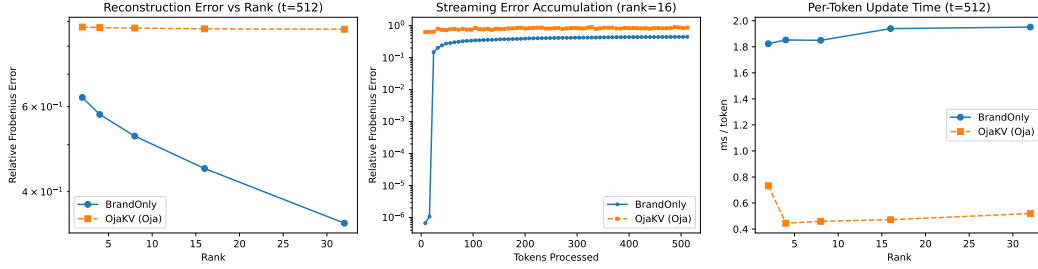


Figure 1: BrandOnly-vs-OjaKV ablation (no freeze). Left: reconstruction error vs rank at $T = 512$. Center: streaming error accumulation at rank 16 over 512 tokens. Right: per-token update time vs rank at $T = 512$.

projections compound; BrandOnly’s EYM guarantee holds at each step (it never freezes in this ablation), giving a consistently lower and flatter error trajectory.

4.2 TKV Long-Context Reconstruction vs OjaKV

The ablation above shows BrandOnly’s warmup mechanism is accurate, but Brand’s update cannot run forever at long context: the $13.4\times$ update-time gap at $T = 16,000$ (Section 2.5) is exactly why TKV freezes after a bounded warmup. We now evaluate TKV itself — warmup through $T_0 = 512$ tokens via Brand updates, then frozen — against OjaKV (which streams throughout) at long context. K/V matrices here are synthetic, with singular values decaying as $1/\sqrt{i+1}$ across the $d = 64$ feature dimensions, allowing controlled sweeps to $T = 32,000$ tokens without the cost of extracting that much real Mistral-7B context.

TKV wins at every (T, R) tested, with the advantage growing at higher rank (full numbers in Appendix A); Figure 2 shows the trend visually.

TKV’s error is nearly constant past the freeze point (e.g., at $R = 16$: $0.537 \rightarrow 0.548 \rightarrow 0.549$ from $T = 1\text{K}$ to 32K) despite receiving no further SVD updates, while OjaKV’s error continues to be dominated by basis staleness.

Decode-phase latency. Using real Mistral-7B KV activations with a shared 256-token warmup, TKV’s frozen-phase update costs 0.19ms per decode token at ranks 8–32 — $2.6\text{--}3.0\times$ faster than OjaKV and roughly $10\text{--}11\times$ faster than continuing to run Brand’s update (BrandOnly), which is exactly the cost TKV’s freeze avoids. At rank 64 this advantage

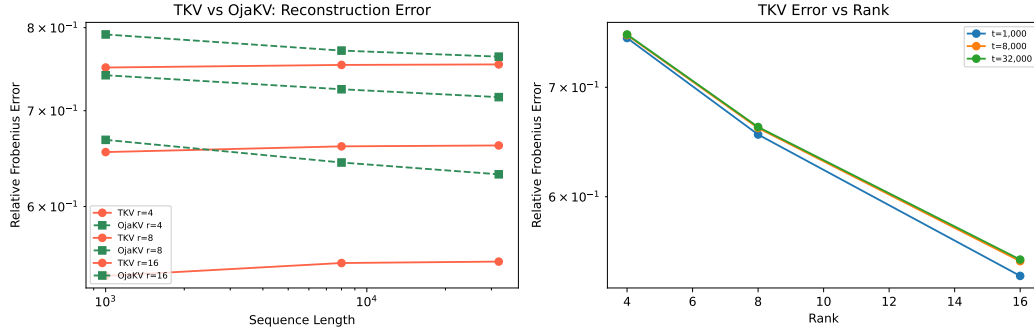


Figure 2: Left: TKV vs OjaKV reconstruction error vs sequence length, by rank. Right: TKV error vs rank, by sequence length. TKV’s error stays essentially flat as T grows past the $T_0 = 512$ freeze point, while OjaKV’s drifting basis degrades.

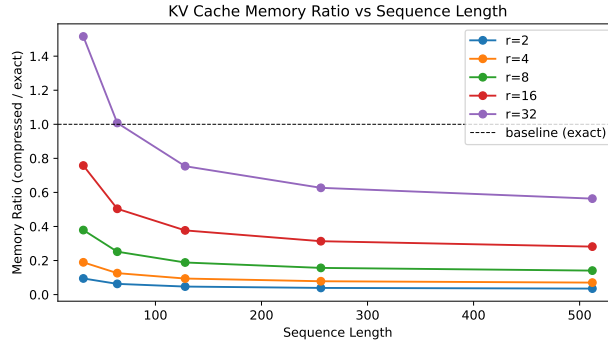


Figure 3: Memory compression ratio vs sequence length at various ranks. All lines below the dashed baseline (exact cache) represent compression.

reverses (TKV 0.77 ms vs. OjaKV’s 0.57 ms); we lack a confident mechanistic explanation
and flag it as an open observation (Section 6) rather than a settled result (full per-rank
numbers in Appendix A).

4.3 Memory Compression

At $T = 512$, $d = 64$, the memory ratio (compressed/dense) is 0.070/0.141/0.282/0.563 for
 $R = 4/8/16/32$ — at rank 16, this corresponds to 28.2% of the dense cache memory, where
the BrandOnly ablation (Section 4.1) shows a $1.95\times$ reconstruction advantage over OjaKV
at the same rank ($T = 512$, Table 1). At rank 32 (56.3% of dense memory), the Halko cold-
start ablation (Section 4.4) is effectively lossless on Mistral-7B too (ratio 0.999, Appendix A).
At the scale TKV targets, the same storage format gives a much larger absolute saving: at
 $T = 128K$ tokens, a TKV rank pyramid (average rank 15) gives a $4.3\times$ compression at
GPT-2 scale that holds consistently from $T = 1K$ to 128K (full breakdown in Appendix A).
On Mistral-7B’s real architecture (32 layers, 8 KV heads, $d = 128$), the same pyramid
gives an even larger $8.5\times$ compression at the same scale (3.94 GB versus 33.55 GB at 128K
tokens, Table 2) — grouped-query attention caches fewer KV heads per layer than GPT-2’s
multi-head attention, so TKV’s per-KV-head compression compounds into a larger overall
saving on the architecture modern long-context models actually use.

Table 2: Mistral-7B-scale memory (32 layers $\times$ 8 KV heads, $d = 128$, float32) at long context, full KV cache vs. a uniform-rank OjaKV ($R = 8$) vs. a TKV rank pyramid (PyramidKV-style layer ranks, average rank 15, as used in Table 3). Compression vs. full is higher than at GPT-2 scale (Appendix A) and holds steady from 1K to 128K tokens.

Tokens	Full KV	OjaKV ($R=8$)	TKV pyramid	Compression
1,000	0.262 GB	0.018 GB	0.035 GB	$7.6\times$
8,000	2.097 GB	0.133 GB	0.250 GB	$8.4\times$
32,000	8.389 GB	0.526 GB	0.987 GB	$8.5\times$
128,000	33.554 GB	2.099 GB	3.936 GB	$8.5\times$

Table 3: Perplexity, exact cache vs. a TKV rank pyramid (avg. rank 15) vs. the same pyramid schedule applied to OjaKV, at both model scales. TKV’s frozen basis stays close to exact at both scales (beating it on GPT-2, landing slightly above it on Mistral-7B) while OjaKV’s drifting basis collapses, more severely at the larger scale.

	Exact	TKV pyramid	OjaKV pyramid
GPT-2 small	129.62	89.59 ($0.691\times$)	756.18 ($5.834\times$)
Mistral-7B	19.39	20.08 ($1.036\times$)	1167.11 ($60.191\times$)

230 4.4 Perplexity (Mistral-7B)

231 We evaluate two configurations on Mistral-7B (32 layers, 8 KV heads via grouped-query
 232 attention, $d = 128$), measuring perplexity on a small held-out set of technical sentences. We
 233 note that this is a proof-of-concept evaluation; results on WikiText-2 or PG-19 are needed
 234 to make strong claims about end-to-end perplexity (Section 6).

235 Halko cold-start, uniform rank (ablation). We first measure a single Halko cold-start SVD
 236 per forward call with no Brand streaming and no freeze, at a single uniform rank applied to
 237 every layer — this isolates the cold-start mechanism’s lossiness on its own, since streaming
 238 Brand correctness is already validated on real activations by Section 4.1.

239 Exact perplexity is 19.39; full per-rank numbers are in Appendix A and Figure 4. At ranks 8
 240 and 16, perplexity is far above exact ($14.2\times$ and $4.8\times$ worse); the knee falls between rank 16
 241 and rank 32, reaching lossless perplexity at 50.2% of dense memory ($T = 128$). This knee
 242 is sharper and shifted higher than GPT-2’s (lossless by rank 32 of $d = 64$), since Mistral’s
 243 larger head dimension means a fixed absolute rank covers a smaller subspace fraction. At
 244 rank 64 the memory ratio exceeds 1.0, underscoring that compression only pays off when
 245 $R \ll \min(T, d)$.

246 TKV, per-layer rank pyramid. We next apply TKV’s freeze mechanism with PyramidKV-
 247 style layer ranks (higher rank for early layers, lower for later ones, average rank 15) and
 248 compare against the same pyramid schedule applied to OjaKV.

249 On a 7B-parameter model, TKV’s frozen rank-15 basis is close to lossless (3.6% degradation)
 250 at $8.5\times$ memory compression (Table 2), while OjaKV with the identical rank budget is $60\times$
 251 worse than exact — the drifting-basis failure mode (Section 4.1) compounds even more
 252 severely here than on GPT-2 ($5.8\times$, Table 3), consistent with drift compounding across
 253 more layers (32 vs. 12) before reaching the language-modeling head. Notably, on GPT-2
 254 small TKV’s pyramid actually beat the exact cache (a 31% improvement) while on Mistral-
 255 7B it lands slightly above exact instead — evidence that the “free regularization” effect is
 256 at least partly a small-model artifact, while the dramatic margin over OjaKV is the robust
 257 finding that holds at both scales.

258 4.5 Importance-Weighted Cold Start: Evaluation

259 To evaluate importance-weighted initialization, we construct a 128×64 key matrix where
 260 the first 32 tokens share a rank-4 subspace (mimicking semantically coherent high-attention

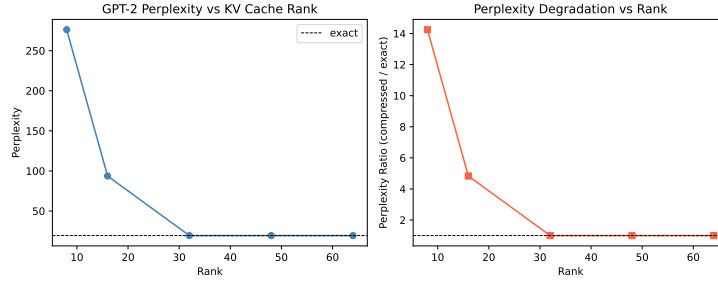


Figure 4: Mistral-7B perplexity (left) and perplexity ratio vs exact (right) as a function of the Halko cold-start ablation’s uniform rank (full numbers in Appendix A). The sharp knee between rank 16 and rank 32 identifies the minimum useful rank for this model geometry.

Table 4: Reconstruction error on important tokens (first 32 of 128) with and without importance weighting using oracle weights. Error reduction is 90–95% at rank 8–16; real attention-derived weights will yield smaller but still substantial gains.

Rank	Uniform error	Weighted error	Reduction
4	0.598	0.495	17.2%
8	0.269	0.014	94.9%
16	0.111	0.008	93.0%
32	0.045	0.004	90.5%

tokens) and the remaining 96 are random noise, and apply oracle importance weights (10.0 for important tokens, 0.1 for noise) derived from ground-truth knowledge of which tokens matter rather than EMA attention statistics — an upper bound on what real attention-derived weights can achieve in practice.

At rank 4, both methods struggle because the rank budget is below the true rank-4 subspace of important tokens plus noise interference; at rank ≥ 8 , importance weighting reduces important-token error by more than $10\times$, suggesting that EMA attention-weight statistics available during generation can dramatically improve re-compression quality when context windows are refreshed.

5 Related Work

Offline KV compression. SVDq (Anonymous, 2025c), xKV (Anonymous, 2025d), KVTC (Anonymous, 2026), and KQ-SVD (Anonymous, 2025a) all apply batch or cross-layer SVD with offline calibration (per-prompt, cross-layer, or attention-matrix decomposition respectively), achieving strong compression but incompatible with streaming generation.

Online KV compression. OjaKV (Anonymous, 2025b) is the closest prior work: Oja’s rule applied token-by-token to maintain an approximate, calibration-free PCA basis with no per-token optimality guarantee. LoRC (Anonymous, 2024a) uses truncated SVD per layer offline. StreamingLLM (Xiao et al., 2023) keeps only sink tokens and a recent window in full precision, orthogonal to low-rank compression; TKV composes directly with this idea (TKV+Sink, full evaluation in Section 6).

Randomized linear algebra. Halko’s framework (Halko et al., 2011) is standard for large-scale randomized SVD, and Brand’s update (Brand, 2006) is well-known in numerical linear algebra but, to our knowledge, not previously applied to KV cache compression. The Φ -bridge’s use of a dense Gaussian matrix satisfying both Johnson-Lindenstrauss and Halko’s sub-Gaussian row requirement (vs. the structured hashing of the count-min sketch (Cormode & Muthukrishnan, 2005)) is a connection novel to this work.

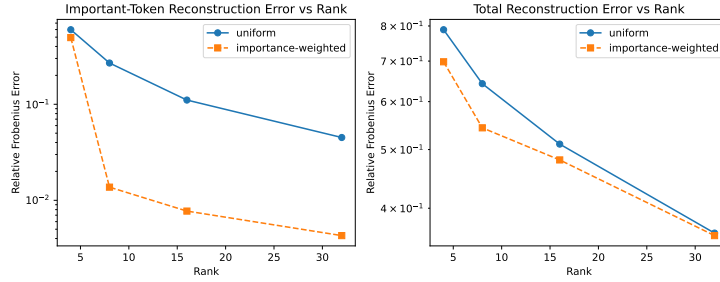


Figure 5: Left: important-token reconstruction error vs rank. Right: total reconstruction error vs rank. Importance weighting provides near-lossless recovery of high-attention tokens at rank ≥ 8 .

287 6 Limitations and Future Work

288 Scope of evaluation. Results cover GPT-2 small and Mistral-7B; a single (layer, KV-head)
 289 pair for the headline ablation, broadened to a 12-configuration sweep (Appendix A) and a
 290 pyramid schedule across all layers/heads for perplexity and long-context. Llama, Qwen, real
 291 (non-synthetic) long-context activations, and a systematic pyramid-schedule sweep across
 292 scales remain open.

293 Evaluation gaps. Importance weighting uses oracle rather than real EMA attention weights;
 294 long-context benchmarks use synthetic, non-shifting statistics, so a pathological distribution
 295 shift after the warmup window T_0 is not yet evaluated empirically (Proposition 1 bounds
 296 this risk theoretically). Perplexity uses a small held-out set rather than full WikiText-2,
 297 and TKV+Sink and dynamic rank scheduling are implemented but not yet benchmarked
 298 end-to-end.

299 Performance. BrandOnly is $3.7\text{--}4.1\times$ slower than OjaKV during warmup; once frozen, TKV
 300 is $2.6\text{--}3.0\times$ faster at ranks 8–32 but reverses at rank 64 for reasons we do not yet understand.
 301 A GPU evaluation is required before deployment.

302 7 Conclusion

303 We presented TKV, an online KV cache compression system combining the Φ -bridge (Gaus-
 304 sian sketch reused as Halko range-finder), Halko cold start and Brand’s exact rank-1 update
 305 for a bounded warmup window, and a freeze transition that holds per-token cost at $\mathcal{O}(R \cdot d)$
 306 indefinitely thereafter — to our knowledge, the first online KV compression method with
 307 a per-token EYM optimality guarantee through warmup, followed by a freeze transition
 308 that avoids Brand’s update cost from compounding at long context. On Mistral-7B’s real
 309 architecture, a TKV rank pyramid reaches $8.5\times$ memory compression at 128K tokens while
 310 landing within 3.6% of exact-cache perplexity, versus OjaKV’s $60\times$ degradation under the
 311 identical rank budget — a margin that holds (and on GPT-2 small, reverses in TKV’s favor)
 312 at both model scales we tested.

313 The core insight is that exact rank-1 updates (Brand) and randomized warm-up (Halko)
 314 are compatible during a bounded window, and that freezing the resulting basis afterward
 315 — rather than continuing to update it exactly forever, or letting it drift as in Oja’s rule —
 316 is what makes the combination viable at long context. The Φ -bridge unifies this pipeline
 317 with the importance-tracking data structure at no additional cost.

318 References

319 Anonymous. Lorc: Low-rank compression for transformer kv caches. In NeurIPS Workshop,
 320 2024a.

- Anonymous. Moment-kv: Importance-weighted kv cache compression via ema attention statistics, 2024b.
- Anonymous. Kq-svd: Optimal low-rank decomposition of the attention matrix, 2025a. arXiv:2512.05916.
- Anonymous. Ojakv: Online kv cache compression via oja’s rule, 2025b. arXiv:2509.21623.
- Anonymous. Svdq: Singular value decomposition with quantization for kv cache, 2025c. arXiv:2502.15304.
- Anonymous. xkv: Cross-layer kv cache compression via singular value decomposition, 2025d. arXiv:2503.18893.
- Anonymous. Kv cache transform coding with randomized svd. In ICLR, 2026.
- M. Brand. Fast low-rank modifications of the thin singular value decomposition. *Linear Algebra and its Applications*, 415(1):20–30, 2006.
- G. Cormode and S. Muthukrishnan. An improved data stream summary: The count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.
- N. Halko, P.-G. Martinsson, and J. A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.
- E. Oja. Simplified neuron model as a principal component analyzer. *Journal of Mathematical Biology*, 15(3):267–273, 1982.
- G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. Efficient streaming language models with attention sinks, 2023. arXiv:2309.17453.

A Appendix: Full Supplementary Numbers

- Decode latency, full numbers (Section 4.2): at $R = 8/16/32/64$, ms/tok is 1.953/2.141/2.052/11.939 (BrandOnly), 0.192/0.188/0.193/0.771 (TKV), 0.498/0.495/0.584/0.568 (OjaKV); TKV error is 0.597/0.536/0.456/0.351.
- BrandOnly vs OjaKV, full layer/KV-head sweep (12 configurations, layers 0, 8, 16, $24 \times$ KV heads 0, 3, 6 of Mistral-7B, summarized in Section 4.1): averaged over the 4 KV-head settings at each layer, BrandOnly’s advantage over OjaKV ranges from $1.61 \times$ ($T=512, R=8$) to $7.67 \times \pm 4.59 \times$ ($T=64, R=32$, the configuration with the highest head-to-head variance) and dominates OjaKV at every one of the 12 configurations tested, consistent with the single-head result in Table 1.
- GPT-2-scale memory compression (12 layers $\times$ 12 heads, $d = 64$, float32; counterpart to Table 2): full KV/OjaKV($R=8$)/TKV-pyramid memory is 0.074/0.010/0.018 GB at 1K tokens, 0.590/0.074/0.139 GB at 8K, 2.359/0.296/0.554 GB at 32K, and 9.437/1.180/2.213 GB at 128K — $4.0\text{--}4.3 \times$ compression throughout, lower than Mistral-7B’s $8.5 \times$ since GPT-2’s multi-head attention caches more KV heads per layer.
- Long-context reconstruction (Figure 2, Section 4.2): across $T = 1,000/8,000/32,000$ and $R = 4/8/16$, TKV’s relative Frobenius error ranges 0.537–0.754 versus OjaKV’s 0.632–0.791 over the same grid — TKV wins at every (T, R) tested, with the gap widening at higher rank. (Halko cold-start perplexity at rank 48, the one value not already given in Section 4.4: 19.37, a 0.753 memory ratio.)